

The Direct Answer

Neither ECC nor Prime Agent "takes care" of running your MCP servers automatically. You still need an MCP host/bridge running inside your local network or Termux environment to manage them. [1, 2]

Harnesses like ECC and Prime Agent are **MCP Clients**—they are the "consumers" of the tools. Your individual GitHub servers (Perplexity, Supabase, Linear, Playwright, Semgrep) are standalone programs that speak a specific language called **JSON-RPC**. They cannot talk to an agent unless an **MCP Host** boots them up, assigns them ports, and translates their inputs/outputs. [3, 4, 5, 6]

1. Why You Need an MCP Host (The Bridge)

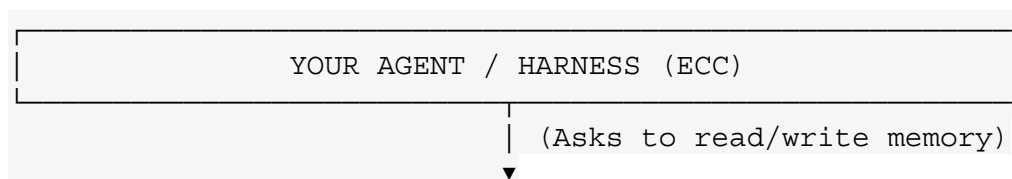
Because you are running on-device open-weights models (Qwen 3.5 0.8B/9B), these models have absolutely no native concept of what an "MCP server" is.

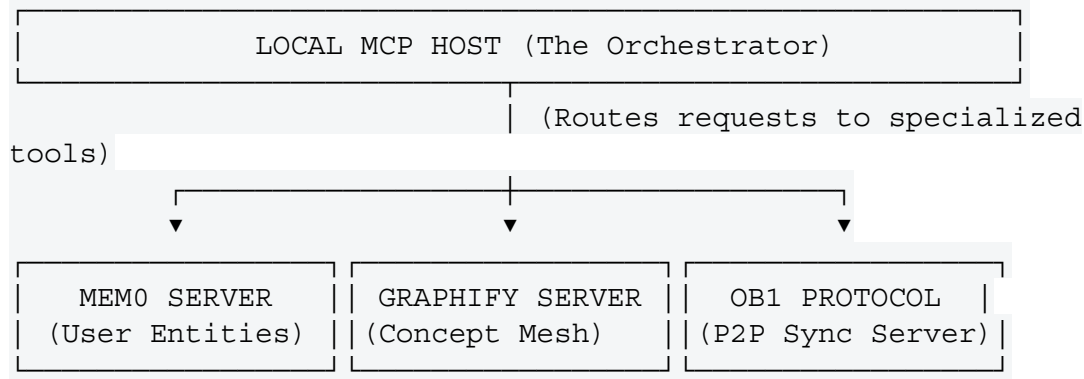
- **The Problem:** If Qwen 3.5 wants to search the web using your Perplexity substitution tool, it can only output a text request. It cannot physically touch your network card or execute code.
- **The Solution:** You need a central **MCP Host (like Node-RED, Smithery, or a simple Python script)** running locally. [7, 8]
- **The Workflow:**
 1. Qwen 3.5 says: *"I need to run a Perplexity search."*
 2. Your **Harness (ECC/Prime)** catches that intent.
 3. The Harness forwards it to the **MCP Host**.
 4. The **MCP Host** physically wakes up the Perplexity GitHub script, runs the search, grabs the results, and passes the clean text back up to the model. [9, 10]

2. How Your Specific Memory Layer Plugs into MCP

Since you are mapping out a highly advanced, multi-tool memory system (mem0 + OmniRoute + ob1 + graphify + LLM Wiki), **MCP is the actual glue that connects them.** [11]

Instead of trying to hardcode mem0 or your Reasoning Bank directly into your Kotlin app or your model's context window, you should wrap your memory tools as **Custom MCP Servers**.





- **The Memory-as-a-Tool Setup:** You create one MCP configuration file. In it, you register mem0 as a tool called `get_long_term_memory` and graphify as a tool called `query_knowledge_graph`.
- **How it Saves Tokens:** When you prompt your system, **OmniRoute** looks at the request. Instead of stuffing your whole Obsidian vault into the model, your agent calls the `query_knowledge_graph` MCP server. The server fetches *only* the specific relationships you need, keeping your Qwen 3.5 context window incredibly lean and fast on your Hexagon NPU. [12]

3. The "No Subscription" Setup for Termux / Home

To keep everything local, lightweight, and under your control, here is exactly how to handle the MCP hosting without paying a dime or using heavy cloud tools:

On Your Phone (Termux)

You can use a simple, open-source python tool you likely already have forked, or use the standard Node-based mcp-cli or a Python gateway to host your local servers. [13, 14]

- You will create a single `mcp_config.json` file in Termux.
- This file lists your local paths to Semgrep, your Perplexity web search substitution, and your Supabase connection string.
- When your Kotlin UI Foreground Service wakes up, it initializes this local host bridge, allowing your dual Qwen models to call them instantly.

On Your Home Node

When you are at home running the Gemma 4 12B task loop, **Prime Agent** can interact with your MCP servers natively because it runs a persistent **IPython kernel**. You can write a tiny Python utility script that imports your MCP servers as raw Python functions directly into Prime Agent's execution pipeline. [15]

Summary of What to Do Right Now

1. **Don't worry about coding the servers yet.** Keep your GitHub forks of Perplexity, Supabase, Playwright, and Semgrep in their folders.
2. **Understand the role:** ECC and Prime Agent will be the "bosses" that read your memory layouts. Your MCP setup will be the "switchboard" that lets them talk to each other. [16]

Since you are mapping out how your local daemons pass screen-captures and memories to this inference loop, it helps to know how the raw data moves. For your on-device local host setup, are you planning to pass your data payloads (like screen arrays or compressed prompt contexts) as **standard JSON strings over local network ports**, or are you planning to write them to **shared local file files (/sdcard/cache)** that your daemons read dynamically?

- [1] <https://www.datacamp.com>
- [2] <https://www.twingate.com>
- [3] <https://tyk.io>
- [4] <https://medium.com>
- [5] <https://www.ksred.com>
- [6] <https://skywork.ai>
- [7] <https://workos.com>
- [8] <https://www.itsfullofstars.de>
- [9] <https://www.busybrain.pub>
- [10] <https://medium.com>
- [11] <https://www.reddit.com>
- [12] <https://support.bluebeam.com>
- [13] <https://ruslanmv.com>
- [14] <https://www.reddit.com>
- [15] <https://www.primeintellect.ai>
- [16] <https://www.markifact.com>